

Surprise Quiz–1 : Solutions

1. Identify which of the following statements (one or more) correctly describe the considerations and characteristics of designing “good” algorithms:
 - A. Algorithm design is a deterministic process governed by a single, universal formula that guarantees efficiency for any given problem.
 - B. The primary trade-off in algorithm design often occurs between Simplicity (ease of coding/debugging) and Efficiency (speed and space usage).
 - C. If a problem occurs very frequently or involves a massive dataset, a designer should prioritize Efficiency over Simplicity, even if the resulting algorithm is more complex.
 - D. While standard paradigms exist, every new problem typically requires a fresh approach, key observations, and the ability to ask the right questions.
 - E. If an efficient algorithm does not currently exist for a practically relevant problem, it is considered theoretically unimportant.

Solution: B, C, D

Explanation:

- **A (False):** There is no single universal formula that guarantees the most efficient algorithm for every problem. Algorithm design depends on the problem constraints and structure.
- **B (True):** A common trade-off in algorithm design is between *Simplicity* (easy to implement/debug) and *Efficiency* (time and memory usage).
- **C (True):** If a problem is frequent or involves large input sizes, efficiency becomes more important, even if the algorithm becomes more complex.
- **D (True):** Even though standard paradigms exist (e.g., greedy, divide-and-conquer, dynamic programming), new problems often require fresh insights and key observations.
- **E (False):** A problem can still be theoretically important even if no efficient algorithm is known (for example, many NP-hard problems are practically and theoretically significant).

2. Consider the following pseudocode used to search for an element x in a rotated sorted array $A[0 \dots n-1]$ containing distinct integers. Fill in the blank in the pseudocode below.

```

low = 0, high = n-1
while (low <= high):
    mid = (low + high) / 2

    if A[mid] == x:
        return mid

    if _____ :
        if A[low] <= x < A[mid]:
            high = mid - 1
        else:
            low = mid + 1
    else:
        if A[mid] < x <= A[high]:
            low = mid + 1
        else:
            high = mid - 1
return -1

```

Solution: $A[\text{low}] \leq A[\text{mid}]$

Explanation:

In a rotated sorted array containing distinct integers, at least one half of the array is always sorted. The problem asks for the correct condition to determine whether the left half of the array is sorted.

The correct condition to check whether the left half is sorted is:

$$A[\text{low}] \leq A[\text{mid}]$$

For example, consider the rotated sorted array:

$$[4, 5, 1, 2, 3]$$

Here, the left half $[4, 5, 1]$ is not sorted, while the right half $[1, 2, 3]$ is sorted. Accordingly,

$$A[\text{low}] > A[\text{mid}]$$

and the algorithm correctly identifies that the right subarray is sorted.

Correct answer: $A[\text{low}] \leq A[\text{mid}]$

3. You need to count the number of nodes in a Binary Search Tree (BST) that have a value strictly greater than x . Which one of the following recursive strategies correctly utilizes the BST property to prune the search?

- A. If the current node's value is $> x$, count the current node and only recurse into the left subtree.
- B. If the current node's value is $\leq x$, ignore the current node and its left subtree, then recurse into the right subtree.
- C. Perform a standard level-order traversal and check every node individually against x .
- D. If the current node's value is $\leq x$, recurse into both the left and right subtrees to ensure no nodes are missed.

Solution: B

Explanation:

- In a BST, all values in the **left subtree** are **less than or equal to** the current node's value.
- All values in the **right subtree** are **greater than** the current node's value.
- If the current node value is $\leq x$, then the current node and its left subtree cannot contain any value strictly greater than x .
- Therefore, we can **prune** the left subtree and recurse only into the **right subtree**.

4. The naive recursive approach for Fibonacci is often used to demonstrate “Simplicity”. What is the Big-O time complexity of this approach?

Solution: $O(2^n)$

Explanation:

- The naive recursive Fibonacci algorithm is defined as:

$$F(n) = F(n - 1) + F(n - 2)$$

- Each call to $F(n)$ makes **two recursive calls**: $F(n - 1)$ and $F(n - 2)$.
- This forms a recursion tree where the number of function calls grows exponentially with n .
- Hence, the running time is exponential and is bounded by:

$$O(2^n)$$

- Simply you can try the complexity by taking some example and find using the recursion

5. If $f(n) = O(g(n))$ and $f(n) = \Theta(h(n))$, then what is true about the relationship between $g(n)$ and $h(n)$?

- A. $h(n) = o(g(n))$
- B. $h(n) = \Theta(g(n))$
- C. $h(n) = \omega(g(n))$
- D. $h(n) = O(g(n))$

Solution: D

Explanation:

Premise 1: $f(n) = \Theta(h(n))$

$f(n)$ grows at the same asymptotic rate as $h(n)$.

$$c_1 h(n) \leq f(n) \leq c_2 h(n) \quad \forall n \geq n_1$$

$f(n)$ and $h(n)$ are interchangeable up to constant factors.

Premise 2: $f(n) = O(g(n))$

$g(n)$ is asymptotically upper bound on $f(n)$.

Combine the two:

$$\begin{aligned} f(n) &\geq c_1 h(n) \\ f(n) &\leq c_3 g(n) \\ c_1 h(n) &\leq c_3 g(n) \\ h(n) &\leq \frac{c_3}{c_1} g(n) \\ h(n) &= O(g(n)) \end{aligned}$$

6. Consider the maximum subarray sum problem on an array $A[1..n]$.

Let

$$S(i) = \text{maximum subarray sum in } A[1..i].$$

A student proposes the following recurrence:

$$S(i) = \max(S(i-1), A[i]).$$

Which of the following statements about the correctness and the time complexity of the solution are correct?

(One or more options may be correct.)

- A. The proposed solution runs in linear time.
- B. The proposed solution is correct because it considers all possible subarrays.
- C. The proposed solution is incorrect because it does not consider all possible subarrays ending at index i .
- D. The proposed solution runs in quadratic time.

Note: A subarray is a contiguous part of an array. A subarray sum is the sum of all the elements in that subarray.

Solution: A,C

Explanation:

Time Complexity:

Recurrence: $T(n) = T(n-1) + c$

Where c is a constant.

Note that we are only doing constant time work at each index.

This gives us a linear time complexity for the proposed solution.

Correctness:

The proposed solution is incorrect.

$S(i)$ = maximum sum over all subarrays that lie entirely within indices 1 to i .

The optimal subarray:

- may end at index i , or
- may end earlier (within $1 \dots i-1$).

Thus, when moving from $i-1$ to i , we must consider subarrays that end at i as well.

The proposed recurrence is incorrect because it only considers:

- (a) The best subarray entirely in $A[1 \dots i-1]$
- (b) The single element subarray $[i]$

What it misses:

It does not consider subarrays that:

- end at index i , and
- start at some index $k < i$

Thus, it ignores subarrays like:

$$A[k] + A[k + 1] + \dots + A[i]$$

Therefore, the recurrence is wrong.

Correct solution (Kadane's Algorithm):

$$\text{dp}[i] = \max(A[i], S(i - 1) + A[i])$$

7. A set of activities has the following start and finish times:

Activity	Start Time	End Time
A	1	5
B	3	9
C	4	7
D	6	10
E	8	12
F	11	13
G	2	6
H	9	11

What is the minimum number of rooms required to schedule all activities such that no two overlapping activities are assigned to the same room?

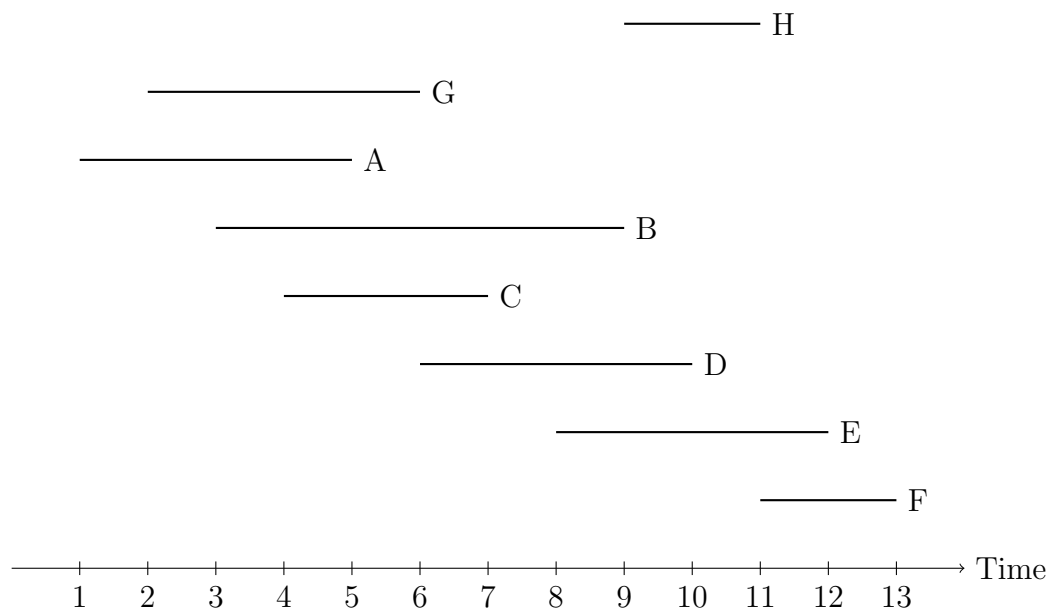
- A. 3
- B. 4
- C. 5
- D. 6

Solution: B

Explanation:

We are given activities with start and end times:

Interval Diagram



Observation

At time $t = 4$, activities A, B, C , and G overlap simultaneously. Hence, the minimum number of rooms required equals the maximum overlap.

4 rooms are required (Option B)

8. Choose all the options (one or more) that are correct about Big-Oh Notation.

A. $3n + 1 = O(n^2)$

B. $n^2 + 5n + 7 = O(n^2)$

C. $\log n = O(n^2)$

D. $2^n = O(n^2)$

Solution: A,B,C

Explanation:

$f(n) = O(g(n))$ if and only if there exists constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Big-Oh gives an upper bound.

A. $3n + 1 = O(n^2)$ because for all $n \geq 1$.

Choose: $c = 4$, $n_0 = 1$

$$3n + 1 \leq 4n \leq 4n^2 \text{ for all } n \geq 1$$

Linear functions are always bounded by quadratic functions.

B. $n^2 + 5n + 7 = O(n^2)$

Choose: $c = 13$, $n_0 = 1$

$$n^2 + 5n + 7 \leq 13n^2 \text{ for all } n \geq 1$$

Lower order terms do not affect Big-Oh classification.

C. $\log n = O(n^2)$

For sufficiently large n , $\log n \ll n^2$

Choose: $c = 1$, $n_0 = 2$

$$\log n \leq n^2 \text{ for all } n \geq 2$$

Any polylogarithmic function is in $O(n^k)$ for any $k > 0$.

D. $2^n = O(n^2)$ is incorrect

Exponential functions grow strictly faster than any polynomial.

Formally: $\lim_{n \rightarrow \infty} \frac{2^n}{n^2} = \infty$.

There is no constant c such that $2^n \leq cn^2$ for all large n .

Exponential growth cannot be upper bounded by polynomial growth.

9. Consider the following array sorted in ascending order:

[3, 8, 15, 22, 31, 44, 56]

How many **comparisons (with array elements)** are required in the **worst case** to search for an element using **binary search**?

- A. 2
- B. 3
- C. 4
- D. 7

Given the sorted array:

[3, 8, 15, 22, 31, 44, 56]

The size of the array is $n = 7$.

In binary search, the **worst-case number of comparisons** is:

$$\lceil \log_2 n \rceil = \lceil \log_2 7 \rceil = \lceil 2.807 \rceil = 3$$

Solution: B

Explanation:

For example, consider searching for the key 56:

- **Comparison 1:** low = 0, high = 6 $\Rightarrow$ mid = 3, $A[3] = 22$
Since $56 > 22$, search in the right subarray.
- **Comparison 2:** low = 4, high = 6 $\Rightarrow$ mid = 5, $A[5] = 44$
Since $56 > 44$, search in the right subarray.
- **Comparison 3:** low = 6, high = 6 $\Rightarrow$ mid = 6, $A[6] == 56$
Element found.

Thus, the element 56 is found after **3 comparisons**, which is the worst case.

10. Given a sorted array A of size n , a modified binary search is used to find the first occurrence of a key x when duplicates are present.

Which of the following statement(s) is/are **correct**? _____

- A. The time complexity remains $O(\log n)$
- B. The time complexity becomes $\theta(n)$
- C. The algorithm terminates as soon as $A[\text{mid}] = x$
- D. On finding $A[\text{mid}] = x$, the search continues in the left subarray
- E. On finding $A[\text{mid}] = x$, the search continues in the right subarray

Solution: A,D

Explanation:

We are given a sorted array that may contain duplicate elements, and we are required to find the first occurrence of a key x . A modified binary search is used, where upon finding x at the middle index, the search continues in the left subarray to ensure that the first occurrence is identified.

The following pseudocode describes the algorithm.

```
low = 0
high = n - 1
result = -1

while (low <= high):
    mid = (low + high) / 2

    if A[mid] == x:
        result = mid
        high = mid - 1    // continue searching left
    else if A[mid] < x:
        low = mid + 1
    else:
        high = mid - 1

return result
```

Now let us analyze each statement:

- (a) The time complexity remains $O(\log n)$.
Correct. The search space is halved at each step.
- (b) The time complexity becomes $\Theta(n)$.
Incorrect. The algorithm does not perform linear scanning.
- (c) The algorithm terminates as soon as $A[\text{mid}] = x$.
Incorrect. To find the first occurrence, further searching is required.
- (d) On finding $A[\text{mid}] = x$, the search continues in the left subarray.
Correct. This ensures the first occurrence is found.

- (e) On finding $A[\text{mid}] = x$, the search continues in the right subarray.
Incorrect. This would be used to find the last occurrence.

Correct options: (a) and (d)

11. We are given 9 tasks $T_1, T_2, \dots, T_9$. The execution of each task requires one unit of time. Only one task can be executed at a time. Each task T_i has a profit P_i and a deadline d_i . Profit P_i is earned if the task is completed before the end of the d_i^{th} unit of time.

Task	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9
Profit	15	20	30	18	18	10	23	16	25
Deadline	7	2	5	3	4	5	2	7	3

Are all tasks completed in the schedule that gives maximum profit?

- A. All tasks are completed
- B. T_1 and T_6 are left out
- C. T_1 and T_8 are left out
- D. T_4 and T_6 are left out

Solution: D

Explanation:

We are given 9 tasks $T_1, T_2, \dots, T_9$. Each task requires exactly one unit of time, and only one task can be executed at a time. Task T_i yields profit P_i if it is completed by the end of its deadline d_i .

Task	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8	T_9
Profit	15	20	30	18	18	10	23	16	25
Deadline	7	2	5	3	4	5	2	7	3

Greedy Algorithm

- (a) Sort all tasks in **non-increasing order of profit**.
- (b) Consider tasks in this order and assign each task to the **last available time slot** not exceeding its deadline.
- (c) If no such slot exists, discard the task.

Placing a task as late as possible ensures that earlier slots remain free for tasks with tighter deadlines. If multiple tasks have the same profit, their relative order does not affect the optimal solution.

Tasks Sorted by Profit

Task	Profit	Deadline
T_3	30	5
T_9	25	3
T_7	23	2
T_2	20	2
T_4	18	3
T_5	18	4
T_8	16	7
T_1	15	7
T_6	10	5

Schedule Construction

The maximum deadline is 7, hence 7 time slots are available. Each task is placed in the last free slot not exceeding its deadline.

Time Slot	Task Assigned
1	T_2
2	T_7
3	T_9
4	T_5
5	T_3
6	T_1
7	T_8

Tasks T_4 and T_6 cannot be scheduled because all slots not exceeding their deadlines are already occupied.

Tasks T_4 and T_6 are left out (Option d)

12. The minimum number of comparisons required to find both the **minimum** and the **maximum** of 100 distinct numbers is: _____
- A. 198
B. 150
C. 148
D. 100

Solution: C

Explanation: We are given 100 distinct numbers. The goal is to find both the minimum and the maximum using the minimum possible number of comparisons.

Naive Approach

Finding the minimum requires 99 comparisons and finding the maximum requires another 99 comparisons.

$$\text{Total} = 198 \text{ comparisons}$$

This approach is not optimal.

Tournament Method

- (a) Pair the numbers arbitrarily and compare each pair.
- (b) Record the winner and loser of each comparison.

Analysis

- Step 1: 100 numbers form 50 pairs $\Rightarrow$ 50 comparisons.
- The maximum must be among the 50 winners $\Rightarrow$ 49 comparisons.
- The minimum must be among the 50 losers $\Rightarrow$ 49 comparisons.

Total Comparisons

$$50 + 49 + 49 = 148$$

This is optimal since every element except the maximum must lose at least once and every element except the minimum must win at least once.

148 comparisons (Option C)

13. What is the time complexity improvement that merge sort achieves over naive sorting algorithms using the divide and conquer approach?

- A) From $O(n^3)$ to $O(n^2)$
- B) From $O(n^2)$ to $O(n \log n)$
- C) From $O(n \log n)$ to $O(n)$
- D) From $O(2^n)$ to $O(n^2)$

Solution: B

Explanation:

Divide and conquer can provide a running time improvement from $O(n^2)$ to $O(n \log n)$, as demonstrated by merge sort. The recurrence relation is: $T(n) = 2T(n/2) + O(n)$

By the Master Theorem, this gives us $T(n) = O(n \log n)$.

14. Consider an array $A = [12, 8, 15, 3, 10, 7, 20, 5]$ which is divided for divide-and-conquer inversion counting into:

- Left half $X = [12, 8, 15, 3]$
- Right half $Y = [10, 7, 20, 5]$

Write all the pairs that represent **split inversions** (i in X , j in Y , and $A[i] > A[j]$)?

Solution: $(12, 10)$, $(12, 7)$, $(12, 5)$, $(8, 7)$, $(8, 5)$, $(15, 10)$, $(15, 7)$, $(15, 5)$

Explanation:

Total: 8 split inversions Split inversions are pairs where: $A[i] > A[j]$

15. Complete the pseudocode for merging two skyline outlines (Fill the blank):

Listing 1: MergeOutlines Pseudocode

```
MergeOutlines(Outline1, Outline2)
    i = 1, j = 1
    height_1 = 0, height_2 = 0
    merged = empty list
    last_pushed_height = 0

    while (i <= length(Outline1) OR j <= length(Outline2))
        if (Outline1[i].x < Outline2[j].x)
            height_1 = Outline1[i].y
            height_to_push = _____ // Fill this
            blank

            if (last_pushed_height != height_to_push)
                Push (Outline1[i].x, height_to_push) to
                    merged
                last_pushed_height = height_to_push
            i = i + 1
        else
            // similar for Outline2

    return merged
```

Solution: max(height_1, height_2)

Explanation:

The skyline problem requires us to compute the outline formed by the union of overlapping rectangles. When merging two outlines, at any x-coordinate, the merged height should be the **maximum** of the heights from both outlines.

Algorithm Logic:

- (a) We maintain **height_1** as the current height in Outline1
- (b) We maintain **height_2** as the current height in Outline2
- (c) When we encounter a new x-coordinate from Outline1:
 - Update **height_1** to the new height at that x-coordinate
 - The merged height is **max(height_1, height_2)**
- (d) We only push to the merged list if the height changes (to avoid consecutive duplicate heights)